Count Inversion Merge Sort

$(i < j)$ form an inversion of $arr[i] > arr[j]$

Ex ⇒ $arr = [\overset{0}{2}, \overset{1}{4}, \overset{2}{1}]$

2 swaps ⇒ 1) swap(arr[1], arr[2]) ⇒ $[\overset{0}{2}, \overset{1}{1}, \overset{2}{4}]$

2) swap(arr[0], arr[1]) ⇒ [1, 2, 4]

⇒ [2, 4, 1] ⇒ [2, 1, 4] ⇒ [1, 2, 4]

⇒ $arr = [\overset{0}{2}, \overset{1}{1}, \overset{2}{3}, \overset{3}{1}, \overset{4}{2}]$

① ⇒ $[\overset{0}{1}, \overset{1}{2}, \overset{2}{3}, \overset{3}{1}, \overset{4}{2}]$ 1 swap

② ⇒ $[\overset{0}{1}, \overset{1}{2}, \overset{2}{1}, \overset{3}{3}, \overset{4}{2}] \rightarrow [\overset{0}{1}, \overset{1}{1}, \overset{2}{2}, \overset{3}{3}, \overset{4}{2}]$ 2 swap

③ ⇒ [1, 1, 2, 2, 3] 1 swap

⇒ 4 swaps ok

⇒ $[\overset{0}{2}, \overset{1}{1}, \overset{2}{3}, \overset{3}{1}, \overset{4}{2}]$ ⓑ i < j & ⓒ arr[i] > arr[j]

mid = 2

TC ⇒ $O(n^2)$

Sp ⇒ $O(1)$

Concept ⇒ { 8, 4, 2, 1 }

⇒ (8, 4) (4, 2) (2, 1)

(8, 2) (4, 1)

(8, 1)

```
for(i=0; i<n; i++)
{
  for(j=i+1; j<n; j++)
  {
    if(arr[i] > arr[j])
    {
      cnt++;
    }
  }
}
```

Another approach (Merge Sort)

$\{ \overset{0}{8}, \overset{1}{4}, \overset{2}{2}, \overset{3}{1} \}$

(4, 8) (2, 1)

(8, 4) (2, 1)

8 4 2 1

In merge() ⇒ 4, 8 (i, j) ⇒ temp array [1 | 2 | 4 | 8]

But in inversion we have to count the possible swaps

Swap ⇒ 4, 8 (mid) | 1, 2 (middle+1, end) (arr[i] > arr[j]) 2 times swap (4, 1) (8, 1) need to invert two times

(4, 1)

(8, 1)

swap ⇒ 4, 8 (mid) | 1, 2 (middle+1, end) (arr[i] > arr[j]) 2 times (4, 2) (8, 2) need to swap two times

(4, 2)

(8, 2)

⇒ To calculate count = mid - i + 1 = 1 - 0 + 1 = 2 ②

⇒ mid - i + 1 ⇒ 1 - 0 + 1 = 2 ②

⇒ i → 8 (mid) | 4 (middle+1, end) ⇒ Inversion count ⇒ mid - i + 1 ⇒ 0 - 0 + 1 = 1 ① ⑥

⇒ i → 2 (mid) | 1 (middle+1, end) ⇒ Inversion count ⇒ mid - i + 1 ⇒ 0 - 0 + 1 = 1 ①

6 total inversion ⇒

{ 8, 4, 2, 1 }

(8, 4) (2, 1) ④

8, 4 ① ① 2, 1

Total 6 inversion count while merging

4, 8 | 2, 1 (i, mid, middle, end) ? swap

if(a[i] > a[j])
{
  count += mid - i + 1;
}

Analogy ⇒ ex ⇒ 2, 4, 6 | 1, 8 (i → 2, 4, 6; j → 1, 8) sorted

arr[i] > arr[j] ⇒ 2 > 1

(2, 1) → 1 (mid - i + 1)

because left subarray is sorted so already swap and if 2 > j ⇒ 2 > 1 then it will be also that 6 > 1